clarified and merged quickly.
  The best way to do this is to just start with the high-level sections and fill
  out details incrementally in subsequent MRs.

Just because a blueprint is merged does not mean it is complete or approved.
Any blueprint is a working document and subject to change at any time.

When editing blueprints, aim for tightly-scoped, single-topic MRs to keep
discussions focused. If you disagree with what is already in a document, open a
new MR with suggested changes.

If there are new details that belong in the blueprint, edit the blueprint. Once
a feature has become "implemented", major changes should get new blueprints.

The canonical place for the latest set of instructions (and the likely source
of this file) is here.

Blueprint statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

<!--
This is the title of your blueprint. Keep it short, simple, and descriptive. A
good title can help communicate what the blueprint is and should be considered
as part of any review.
-->

<!--
For long pages, consider creating a table of contents.
The [TOC] function is not supported on docs.gitlab.com.
-->
{{< engineering/design-document-header >}}

For important terms, see glossary.

 Summary

EPSS scores specify the likelihood a CVE will be exploited in the next 30 days. This data may
be used to improve and simplify prioritization efforts when remediating vulnerabilities in a
project. EPSS support requirements are outlined in the EPSS epic along with an overview of
EPSS. This document focuses on the technical implementation of EPSS support.

EPSS scores may be populated from the EPSS Data page or through their provided API. Ultimately,

EPSS scores should be reachable through the GitLab GraphQL API, as seen on the vulnerability report and details pages, and be filterable and usable when setting policies.

Package metadata database (PMDB, also known as license-db), an existing advisory pull-and-enrichment method, is for this purpose. The flow is as follows:

mermaid
flowchart LR
    A[EPSS Source] -->|Pull| B[PMDB]
    B -->|Process and export| C[Bucket]
    C -->|Pull| D[GitLab Instance]

<!--
This section is very important, because very often it is the only section that
will be read by team members. We sometimes call it an "Executive summary",
because executives usually don't have time to read entire document like this.
Focus on writing this section in a way that anyone can understand what it says,
the audience here is everyone: executives, product managers, engineers, wider
community members.

A good summary is probably at least a paragraph in length.
-->

Motivation

The classic approach to vulnerability prioritization is using severity based on CVSS. This approach provides some guidance, but is too unrefined·more than half of all published CVEs have a high or critical score. Other metrics need to be employed to reduce remediation fatigue and help developers prioritize their work better. EPSS provides a metric to identify which vulnerabilities are most likely to be exploited in the near future. Combined with existing prioritization methods, EPSS helps to focus remediation efforts better and reduce remediation workload. By adding EPSS to the information presented to users, we deliver these benefits to the GitLab platform.

<!--
This section is for explicitly listing the motivation, goals and non-goals of
this blueprint. Describe why the change is important, all the opportunities,
and the benefits to users.

The motivation section can optionally provide links to issues that demonstrate
interest in a blueprint within the wider GitLab community. Links to
documentation for competing products and services is also encouraged in cases
where they demonstrate clear gaps in the functionality GitLab provides.

For concrete proposals we recommend laying out goals and non-goals explicitly,
but this section may be framed in terms of problem statements, challenges, or
opportunities. The latter may be a more suitable framework in cases where the
problem is not well-defined or design details not yet established.
-->

Goals

- Enable users to use EPSS scores on GitLab as another metric for their vulnerability prioritization efforts.
- Provide scalable means of efficiently repopulating recurring EPSS scores to minimize system load.

Phase 1 (MVC)

- Enable access to EPSS scores through GraphQL API.

Phase 2

- Show EPSS scores in vulnerability report and details pages.

Phase 3

- Allow filtering vulnerabilities based on EPSS scores.
- Allow creating policies based on EPSS scores.

<!--
List the specific goals / opportunities of the blueprint.

- What is it trying to achieve?
- How will we know that this has succeeded?
- What are other less tangible opportunities here?
-->

Non-Goals

- Dictate priority to users based on EPSS (or any other metric).

<!--
Listing non-goals helps to focus discussion and make progress. This section is optional.

- What is out of scope for this blueprint?
-->

Proposal

Support EPSS on the GitLab platform.

Following the discussions in the EPSS epic, the proposed flow is:

1. PMDB database is extended with a new table to store EPSS scores.
1. PMDB infrastructure runs the feeder daily in order to pull and process EPSS data.
1. The advisory-processor receives the EPSS data and stores them to the PMDB DB.
1. PMDB exports EPSS data to a new PMDB EPSS bucket.
    - Create a new bucket to store EPSS data.

   - Delete former EPSS data once new data is uploaded, as the old data is no longer needed.
   - Truncate EPSS scores to two digits after the dot.
1. GitLab instances pull data from the PMDB EPSS bucket.
   - Create a new table in rails DB to store EPSS data.
1. GitLab instances expose EPSS data through GraphQL API and present data in vulnerability report and details pages.

mermaid
flowchart LR
   AF[Feeder] -->|pulls| A[EPSS Source]
   AF -->|publishes| AP[Advisory Processor]
   AP -->|stores| DD[PMDB database]
   E[Exporter] -->|loads|DD
   E --> |exports| B[Public Bucket]
   GitLab[GitLab instance] --> |syncs| B
   GitLab --> |stores| GitLabDB

<!--
This is where we get down to the specifics of what the proposal actually is,
but keep it simple! This should have enough detail that reviewers can
understand exactly what you're proposing, but should not include things like
API designs or implementation. The "Design Details" section below is for the
real nitty-gritty.

You might want to consider including the pros and cons of the proposed solution so that they can be
compared with the pros and cons of alternatives.
-->

Design and implementation details

Decisions

- 001: Export all EPSS entries
- 002: Use a new bucket for EPSS data
- 003: Switch from API to CSV file

Important notes

- All EPSS scores get updated on a daily basis. This is pivotal to this feature's design.
- The fields retrieved from the EPSS source are cve, score, percentile. 9 digits after the dot are maintained.
  - To reduce the amount of upserts, based on a spike to check magnitude of change, we will truncate EPSS scores to two digits after the dot.

PMDB

- Create a new EPSS table in PMDB with an advisory identifier and the EPSS score. This includes changing the schema and any necessary migrations.

- Ingest EPSS data into new PMDB table. We want to keep the EPSS data structure as close as possible to the origin so all of the data may be available to the exporter, and the exporter may choose how to process it. Therefore we will save scores and percentiles with their complete values.
- Export EPSS scores in separate bucket.
  - Delete the previous day's export as it is no longer needed after the new one is added.
- Add new pubsub topics to deployment to be used by PMDB components, using existing terraform modules.

GitLab Rails backend

- Create table in rails backend to hold EPSS scores.
- Configure Rails sync to ingest EPSS exports and save to new table.
- Include EPSS data attributes in GraphQL API Occurrence objects.

GitLab UI

- Add EPSS data to vulnerability report page.
- Add EPSS data to vulnerability details page.
- Allow filtering by EPSS score.
- Allow creating policies based on EPSS score.

<!--
This section should contain enough information that the specifics of your change are understandable. This may include API specs (though not always required) or even code snippets. If there's any ambiguity about HOW your proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the blueprint, the rule of thumb here is to provide enough context for people to understand the proposal. As you move forward with the implementation, you may need to add more implementation details to the blueprint, as those may become an important context for important technical decisions made along the way. A blueprint is also a register of such technical decisions. If a technical decision requires additional context before it can be made, you probably should document this context in a blueprint. If it is a small technical decision that can be made in a merge request by an author and a maintainer, you probably do not need to document it here. The impact a technical decision will have is another helpful information - if a technical decision is very impactful, documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images. Diagrams authored in GitLab flavored markdown are preferred. In cases where that is not feasible, images should be placed under images/ in the same directory as the index.md for the proposal.
-->

Alternative Solutions

<!--

It might be a good idea to include a list of alternative solutions or paths considered,
although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

Glossary

- PMDB (Package metadata database, also known as License DB): PMDB is a standalone service (and not solely a database), outside of the Rails application, that gathers, stores and exports packages metadata for GitLab instances to consume. See complete documentation. PMDB components include:
  - Feeder: a scheduled job called by the PMDB deployment to publish data from the relevant sources to pub/sub messages consumed by PMDB processors.
  - Advisory processor: Runs as a Cloud Run instance and consumes messages published by the advisory feeder containing advisory related data and stores them to the PMDB database.
  - PMDB database: a PostgreSQL instance storing license and advisory data.
  - Exporter: exports license/advisory data from the PMDB database to public GCP buckets.
- GitLab database: the database used by GitLab instances.
- CVE (Common Vulnerabilities and Exposures): a list of publicly known information-security vulnerabilities. "A CVE" usually refers to a specific vulnerability and its CVE ID.
- EPSS (Exploit prediction scoring system) score: a score ranging from 0 to 1 representing the probability of exploitation in the wild in the next 30 days of a given vulnerability.
- EPSS score percentile: for a given EPSS score (of some vulnerability), the proportion of all scored vulnerabilities with the same or a lower EPSS score.

SECTION: engineering/architecture/design-documents/epss/decisions/001_export_all_epss.md

Context

PMDB advisories are exported using deltas. New advisories and advisories which have changed since the last export are exported each time the exporter runs.
This can be observed in the PMDB advisory bucket, where exports are organized in directories titled with a timestamp.
Each directory contains the items changed since the preceding directory's timestamp.
This is done to avoid very large updates of data.

EPSS scores specify the probability that a vulnerability will be exploited in the next 30 days.
Therefore, all EPSS scores are updated every day.
This renders the delta approach unhelpful·if almost all values change every day, then the deltas will always be big and will save very little resources.

However, EPSS data is relatively small. There are around 250,000 entries (although this number grows every day) and the raw data doesn't pass 10MB.
The size of a DB table containing all EPSS scores data was estimated as up to 25MB.
It is likely to be smaller. This is not a very significant size to update once a day.

As a result, the MVC approach is to pull and export all EPSS scores each day, rather than using deltas.

Decision

Pull and export all EPSS scores each day, rather than using deltas.

Consequences

The implementation is simpler as we don't need to determine deltas.
The EPSS PMDB bucket will contain just one directory which will be overwritten with every pull, as all scores are updated.

This flow strays from the existing advisories approach of PMDB.

EPSS data grows slowly (as new vulnerabilities are released), so over time we will be exporting more and more data, up to around 30,000 new rows per year.
This should still not be a significant load, but we should ultimately consider a lighter approach.

Alternatives

Using deltas is ultimately preferable to reduce the amount of data transferred and saved in GitLab instances each day.
This option was deferred due to the low impact of the size of EPSS data.
A spike looking into actual data changes in EPSS demonstrated that some manipulation of the data (truncating values to two digits after the dot; not including percentile) allows reducing deltas significantly, and in the future we may implement this approach to support using deltas.

SECTION: engineering/architecture/design-documents/epss/decisions/002_use_new_bucket.md

Context

PMDB exports data to GCP buckets. The data is later pulled by GitLab instances. Advisory data and license data are stored in different buckets. This is sensible, because advisory and license data are not directly related, and rather provide additional information about packages. Data is updated based on deltas·changes from the previous state of the data. Only those changes are saved with each addition to the database.

EPSS data is directly associated with advisories, so a simple solution would be to add it as another directory of data to the existing advisories bucket to keep them grouped together.

- v2/<purltype>/<timestamp>/<sequence>.ndjson contains advisories.
- v2/<purltype>/epss/<timestamp>/<sequence>.ndjson or v2/epss/<timestamp>/<sequence>.ndjson would contain EPSS data.

However, the current advisories bucket is structured based on purltype. Adding an epss data type would couple epss with purltype which is a faulty pairing. (EPSS data corresponds to a

CVE, and a single CVE might affect multiple packages that have different PURL types.) Due to the tight coupling between purltype and the existing advisories bucket, it would be difficult and convoluted to add epss to it.

Furthermore, we could add the EPSS data directly to the relevant advisory JSON objects. However, since EPSS data is updated daily, this would risk frequent exports of large amounts of entire advisories due to many changing EPSS scores. This solution was not chosen to avoid performance issues.

Following extensive discussions on the EPSS epic and discussion during the refinement of PMDB issues, it was initially decided to use the existing bucket as this feels most intuitive and at the time felt a healthier approach. Further discussion during the refinement of the GitLab backend effort led to the decision to use a new bucket, due to the complexity of the coupling of purltype and other, unrelated areas in the monolith. Adding epss to purltype would impact other components and we want to avoid having to work around that. We may want to later simplify these areas and reconsider the bucket structure at a later stage.

Decision

Export EPSS data to a new bucket, rather than exporting it into the existing PMDB advisories bucket.

Consequences

The implementation is simpler than adding a directory to the existing advisories bucket, but may feel less intuitive.
This change require the relevant Terraform changes regarding the provisioning of a new bucket. This should also be addressed in the exporter and the GitLab packagemetadata sync configuration.

Alternatives

One option is to add EPSS data to the advisories bucket as another directory, since they are directly related. This was the initial decision. This would allow us to utilize existing mechanisms and keep related data close. However, EPSS data doesn't fit into the current structure of the advisories bucket. An ideal solution would reconstruct the buckets in a manner more fitting for this approach, but this would be a big effort and is not critical enough.

Another option is to add EPSS data directly into advisory JSON objects. This was rejected due to the update nature of EPSS. EPSS is updated daily, which may lead to having to export many entire advisories due to changes to the EPSS field, greatly increasing daily deltas and performance impact on GitLab instances.

SECTION: engineering/architecture/design-documents/epss/decisions/003_switch_from_api_to_csv_file.md

Context

The EPSS Feeder in PMDB retrieves data from the EPSS source and publishes it via GCP's Pub/Sub. Two options were evaluated:

1. Using the API to fetch data directly
2. Downloading a compressed CSV file containing EPSS data

Initially, we chose the API. However, experience has shown significant stability issues with the API approach.
Specific examples of these issues can be found in this GitLab issue

Decision

Switch from the API to downloading and processing the compressed CSV file for retrieving EPSS data.

Consequences

Positive

- Improved stability of the license-feeder EPSS flow (both production and test environments).
- Single network request instead of multiple API calls.
- Faster processing.

Negative

- Must keep all data in memory while processing them. Given the size of the CSV this is not a problem.
- Can't choose to download only specific parts of the data (though this is not a relevant use case)

Alternatives

Continue using the API with improved error handling and retry mechanisms, but this would add complexity without addressing the root cause of the stability issues.

SECTION: engineering/architecture/design-documents/feature_flags_development/_index.md

{{< engineering/design-document-header >}}

Usage of feature flags become crucial for the development of GitLab. The feature flags are a convenient way to ship changes early, and safely rollout them to wide audience ensuring that feature is stable and performant.

Since the presence of feature is controlled with a dedicated condition, a developer can decide for a best time for testing the feature, ensuring that feature is not enable prematurely.

Challenges

The extensive usage of feature flags poses a few challenges

- Each feature flag that we add to codebase is a ~"technical debt" as it adds a matrix of configurations.
- Testing each combination of feature flags is close to impossible, so we instead try to optimize our testing of feature flags to the most common scenarios.
- There's a growing challenge of maintaining a growing number of feature flags. We sometimes forget how our feature flags are configured or why we haven't yet removed the feature flag.
- The usage of feature flags can also be confusing to people outside of development that might not fully understand dependence of ~"type::feature" or ~"type::bug" fix on feature flag and how this feature flag is configured. Or if the feature should be announced as part of release post.
- Maintaining feature flags poses additional challenge of having to manage different configurations across different environments/target. We have different configuration of feature flags for testing, for development, for staging, for production and what is being shipped to our customers as part of on-premise offering.

Goals

The biggest challenge today with our feature flags usage is their implicit nature. Feature flags are part of the codebase, making them hard to understand outside of development function.

We should aim to make our feature flag based development to be accessible to any interested party.

- developer / engineer
  - can easily add a new feature flag, and configure it's state
  - can quickly find who to reach if touches another feature flag
  - can quickly find stale feature flags
- engineering manager
  - can understand what feature flags her/his group manages
- engineering manager and director
  - can understand how much ~"technical debt" is inflicted due to amount of feature flags that we have to manage
  - can understand how many feature flags are added and removed in each release
- product manager and documentation writer
  - can understand what features are gated by what feature flags
  - can understand if feature and thus feature flag is generally available on GitLab.com
  - can understand if feature and thus feature flag is enabled by default for on-premise installations
- delivery engineer
  - can understand what feature flags are introduced and changed between subsequent deployments
- support and reliability engineer
  - can understand how feature flags changed between releases: what feature flags become enabled, what removed
  - can quickly find relevant information about feature flag to know individuals which might

help with an ongoing support request or incident

Proposal

To help with above goals we should aim to make our feature flags usage explicit and understood by all involved parties.

Introduce a YAML-described feature-flags/<name-of-feature.yml> that would allow us to have:

1. A central place where all feature flags are documented,
1. A description of why the given feature flag was introduced,
1. A what relevant issue and merge request it was introduced by,
1. Build automated documentation with all feature flags in the codebase,
1. Track how many feature flags are per given group
1. Track how many feature flags are added and removed between releases
1. Make this information easily accessible for all
1. Allow our customers to easily discover how to enable features and quickly find out information what did change between different releases

The YAML

yaml
---
name: cidisallowtocreatemergerequestpipelinesintargetproject
introducedbyurl: https://gitlab.com/gitlab-org/gitlab/-/mergerequests/40724
rolloutissueurl: https://gitlab.com/gitlab-org/gitlab/-/issues/235119
group: group::environments
type: development
defaultenabled: false

Reasons

These are reason why these changes are needed:

- we have around 500 different feature flags today
- we have hard time tracking their usage
- we have ambiguous usage of feature flag with different defaultenabled: and different actors used
- we lack a clear indication who owns what feature flag and where to find relevant information
- we do not emphasise the desire to create feature flag rollout issue to indicate that feature flag is in fact a ~"technical debt"
- we don't know exactly what feature flags we have in our codebase
- we don't know exactly how our feature flags are configured for different environments: what is being used for test, what we ship for on-premise, what is our settings for staging, qa and production

Iterations

This work is being done as part of dedicated epic:
Improve internal usage of Feature Flags.
This epic describes a meta reasons for making these changes.

SECTION: engineering/architecture/design-documents/feature_flags_usage_in_dev_and_ops/_index.md

{{< engineering/design-document-header >}}

This blueprint builds upon the Development Feature Flags Architecture blueprint.

Summary

Feature flags are critical both in developing and operating GitLab, but in the current state of the process, they can lead to production issues, and introduce a lot of manual and maintenance work.

The goals of this blueprint is to make the process safer, more maintainable, lightweight, automated and transparent.

Motivations

Feature flag use-cases

Feature flags can be used for different purposes:

- De-risking GitLab.com deployments (most feature flags): Allows to quickly enable/disable a feature flag in production in the event of a production incident.
- Work-in-progress feature: Some features are complex and need to be implemented through several MRs. Until they're fully implemented, it needs to be hidden from anyone. In that case, the feature flag allows to merge all the changes to the main branch without actually using the feature yet.
- Beta features: We might not be confident we'll be able to scale, support, and maintain a feature in its current form for every designed use case (example). There are also scenarios where a feature is not complete enough to be considered an MVC. Providing a flag in this case allows engineers and customers to disable the new feature until it's performant enough.
- Operations: Site reliability engineer or Support engineer can use these flags to disable potentially resource-heavy features in order to the instance back to a more stable and available state. Another example is SaaS-only features.
- Experiment: A/B testing on GitLab.com.
- Worker (special ops feature flag): Used for controlling Sidekiq workers behavior, such as deferring Sidekiq jobs.

We need to better categorize our feature flags.

Production incidents related to feature flags

Feature flags have caused production incidents on GitLab.com (1, 2, 3).

We need to prevent this for the sake of GitLab.com stability.

Technical debt caused by feature flags

Feature flags are also becoming an ever-growing source of technical debt: there are currently 591 feature flags in the GitLab codebase.

We need to reduce feature flags count for the sake of long-term maintainability & quality of the GitLab codebase.

Goal

The goal of this blueprint is to improve the feature flag process by making it:

- safer
- more maintainable
- more lightweight & automated
- more transparent

Challenges

Complex feature flag rollout process

The feature flag rollout process is currently:

- Complex: Rollout issues that are very manual and includes a lot of checkboxes (including non-relevant checkboxes).
  Engineers often don't use these issues, which tend to become stale and forgotten over time.
- Not very transparent: Feature flag changes are logged in several places far from the rollout issue, which makes it hard to understand the latest feature flag state.
- Far from production processes: Rollout issues are created in the gitlab-org/gitlab project (far from the production issue tracker).
- There is no consistent path to rolling out feature flags: we leave to the judgement of the engineer to trade-off between speed and safety. There should be a standardized set of rollout steps.

Technical debt and codebase complexity

The challenges from the Development Feature Flags Architecture blueprint still stand.

Additionally, there are new challenges:

- If a feature flag is enabled by default, and is disabled in an on-premise installation, then when the feature flag is removed, the feature suddenly becomes enabled on the on-premise instance and cannot be rolled backed to the previous behavior.

Multiple source of truth for feature flag default states and observability

We currently show the feature flag default states in several places, for different intended audiences:

GitLab customers

- User documentation:
  List all feature flags and their metadata so that GitLab customers can tweak feature flags on their instance. Also useful for GitLab.com users that want to check the default state of a feature flag.

Site reliability and Delivery engineers

- Internal GitLab.com feature flag state change issues:
  For each change of a feature flag state on GitLab.com, an issue is created in this project.
- Internal GitLab.com feature flag state change logs:
  Filter logs with source: feature and env: gprd to see feature flag state change logs.

GitLab Engineering & Infra/Quality Directors / VPs, and CTO

- Internal Sisense dashboard:
  Feature flag metrics over time, grouped per DevOps groups.

GitLab Engineering and Product managers

- "Feature flags requiring attention" monthly reports:
  Same data as the above Internal Sisense dashboard but for a specific DevOps group, presented in an issue and assigned to the group's Engineering managers.

Anyone who wants to check feature flag default states

- Unofficial feature flags dashboard:
  A user-friendly dashboard which provides useful filtering.

This leads to confusion for almost all feature flag stakeholders (Development engineers, Engineering managers, Site reliability, Delivery engineers).

Proposal

Improve feature flags implementation and usage

- Reduce the likelihood of mis-configuration and human-error at the implementation step
  - Remove the "percentage of time" strategy in favor of "percentage of actors"
- Improve the feature flag development documentation

Introduce new feature flag types

It's clear that the development feature flag type actually includes several use-cases:

- GitLab.com deployment de-risking. YAML value: gitlabcomderisk.
- Work-in-progress feature. YAML value: wip. Once the feature is complete, the feature flag type can be changed to beta
  if there still are some doubts on the scalability of the feature.
- Beta features. YAML value: beta.

Notes:

- These new types replace the broad development type, which shouldn't be used anymore in the future.
- Backward-compatibility will be kept until there's no development feature flags in the codebase anymore.

Introduce constraints per feature flag type

Each feature flag type will be assigned specific constraints regarding:

- Allowed values for the defaultenabled attribute
- Maximum Lifespan (MLS): the duration starting on the introduction of the feature flag (i.e. when it's merged into master).
  We don't introduce a life span that would start on the global GitLab.com enablement (or defaultenabled: true when
  applicable) so that there's incentive to rollout and delete feature flags as quickly as possible.

The MLS will be enforced through automation, reporting & regular review meetings at the section level.

Following are the constraints for each feature flag type:

- gitlabcomderisk
  - defaultenabled must not be set to true. This kind of feature flag is meant to lower the risk on GitLab.com, thus
    there's no need to keep the flag in the codebase after it's been enabled on GitLab.com.
    defaultenabled: true will not have any effect for this type of feature flag.
  - Maximum Lifespan: 2 months.
  - Additional note: This type of feature flag won't be documented in the All feature flags in GitLab
    page given they're short-lived and deployment-related.
- wip
  - defaultenabled must not be set to true. If needed, this type can be changed to beta once the feature is complete.
  - Maximum Lifespan: 4 months.
- beta
  - defaultenabled can be set to true so that a feature can be "released" to everyone in beta with the possibility to disable
    it in the case of scalability issues (ideally it should only be disabled for this reason on specific on-premise installations).
  - Maximum Lifespan: 6 months.
- ops

  - defaultenabled can be set to true.
  - Maximum Lifespan: Unlimited.
  - Additional note: Remember that using this type should follow a conscious decision not to introduce an instance setting.
- experiment
  - defaultenabled must not be set to true.
  - Maximum Lifespan: 6 months.

Introduce a new featureissueurl field

Keeping the URL to the original feature issue will allow automated cross-linking from the rollout
and logging issues. The new field for this information is featureissueurl.

For instance:

yaml

name: aimrcreation
featureissueurl: https://gitlab.com/gitlab-org/gitlab/-/issues/12345
introducedbyurl: https://gitlab.com/gitlab-org/gitlab-foss/-/mergerequests/14218
rolloutissueurl: https://gitlab.com/gitlab-com/gl-infra/production/-/issues/83652
milestone: '16.3'
type: beta
group: group::code review
defaultenabled: true

Streamline the feature flag rollout process

1. (Process) Transition to create rollout issues in the
   Production issue tracker and adapt the
   template to be closer to the
   Change management issue template
   (see this issue for inspiration)
   That way, the rollout issue would only concern the actual production changes (i.e. enablement/disablement
   of the flag on production) and should be closed as soon as the production change is confirmed to work as expected.
1. (Automation) Automate most rollout steps, such as:
    - (Done) Let the author know that their feature has been deployed to staging / canary / production environments
    - (Done) Cross-link actual feature flag state change (from Chatops project) to rollout issues
    - (Done) Let the author know that their defaultenabled: true MR has been deployed to production and that the feature flag can be removed from production
    - Automate the creation of rollout issues when a feature flag is first introduced in a merge request,
      and provide an diff suggestion to fill the rolloutissueurl field (Danger)
    - Check and enforce feature flag definition constraints in merge requests (Danger)

- Provide a diff suggestion to correct the milestone field when it's not the same value as the MR milestone (Danger)
- Upon feature flag state change, notify on Slack the group responsible for it (chatops)
- 7 days before the Maximum Lifespan of a feature flag is reached, automatically create a "cleanup MR" with the group label set, and assigned to the feature flag author (if they're still with GitLab). We could take advantage of the automation of repetitive developer tasks
- Enforce Maximum Lifespan of feature flags through automated reporting & regular review at the section level

1. (Documentation/process) Ensure the rollout DRI stays online for a few hours after enabling a feature flag (ideally they'd enable the flag at the beginning of their day) in case of any issue with the feature flag
1. (Process) Provide a standardized set of rollout steps. Trade-offs to consider include:
   - Likelihood of errors occurring
   - Total actors (users / requests / projects / groups) affected by the feature flag rollout, e.g. it will be bad if 100,000 users cannot log in when we roll out for 1%
   - How long to wait between each step. Some feature flags only need to wait 10 minutes per step, some flags should wait 24 hours. Ideally there should be automation to actively verify there is no adverse effect for each step.

Provide better SSOT for the feature flag default states and current states & state changes on GitLab.com

GitLab customers

- User documentation:
  Keep the current page but add filtering and sorting, similarly to the unofficial feature flags dashboard.

Site reliability and Delivery engineers

We assessed the usefulness of feature flag state change logging strategies and it appears that both internal GitLab.com feature flag state change issues and internal GitLab.com feature flag state change logs are useful for different audiences.

GitLab Engineering & Infra/Quality Directors / VPs, and CTO

- Internal Sisense dashboard:
  Streamline the current dashboard to be more useful for its stakeholders.

GitLab Engineering and Product managers

- "Feature flags requiring attention" monthly reports:
  Make the current reports more actionable by linking to automatically created MRs for removing feature flags as well as improving documentation and best-practices around feature flags.

Iterations

This work is being done as part of dedicated epic:
Improve internal usage of Feature Flags.
This epic describes a meta reasons for making these changes.

Resources

- What Are Feature Flags?
- Feature Flags Best Practices
- Short-lived or Long-lived Flags? Explaining Feature Flag lifespans

---
SECTION: engineering/architecture/design-documents/gitaly_adaptive_concurrency_limit/_index.md

{{< engineering/design-document-header >}}

Summary

Gitaly, a Git server, needs to push back on its clients to reduce the risk of incidents. In the past, we introduced per-RPC concurrency limit and pack-objects concurrency limit. Both systems were successful, but the configurations were static, leading to some serious drawbacks. This blueprint proposes an adaptive concurrency limit system to overcome the drawbacks of static limits. The algorithm primarily uses the Additive Increase/Multiplicative Decrease approach to gradually increase the limit during normal processing but quickly reduce it during an incident. The algorithm focuses on lack of resources and serious latency degradation as criteria for determining when Gitaly is in trouble.

Motivation

To reduce the risk of incidents and protect itself, Gitaly should be able to push back on its clients when it determines some limits have been reached. In the prior attempt, we laid
out some foundations for backpressure
by introducing two systems: per-RPC concurrency limits and pack-objects concurrency limits.

Per-RPC concurrency limits allows us to configure a maximum amount of in-flight requests simultaneously. It scopes the limit by RPC and repository. Pack-objects concurrency limit restricts the concurrent Git data transfer request by IP. One note, the pack-objects concurrency limit is applied on cache misses, only. If this limit is exceeded, the request is either put in a queue or rejected if the queue is full. If the request remains in the queue for too long, it will also be rejected.

Although both of them yielded promising results on GitLab.com, the configurations, especially the value of the concurrency limit, are static. There are some drawbacks to this:

- It's tedious to maintain a sane value for the concurrency limit. Looking at this production configuration, each limit is heavily calibrated based on clues from different sources. When the overall scene changes, we need to tweak them again.
- Static limits are not good for all usage patterns. It's not feasible to pick a fit-them-all value. If the limit is too low, big users will be affected. If the value is too loose, the protection effect is lost.
- A request may be rejected even though the server is idle as the rate is not necessarily an indicator of the load induced on the server.

To overcome all of those drawbacks while keeping the benefits of concurrency limiting, one promising solution is to make the concurrency limit adaptive to the currently available processing capacity of the node. We call this proposed new mode "Adaptive Concurrency Limit".

Goals

- Make Gitaly smarter in push-back traffic when it's under heavy load, thus enhancing the reliability and resiliency of Gitaly.
- Minimize the occurrences of Gitaly saturation incidents.
- Decrease the possibility of clients inaccurately reaching the concurrency limit, thereby reducing the ResourceExhausted error rate.
- Facilitate seamless or fully automated calibration of the concurrency limit.

Non-goals

- Increase the workload or complexity of the system for users or administrators. The adaptiveness proposed here aims for the opposite.

Proposal

The proposed Adaptive Concurrency Limit algorithm primarily uses the Additive Increase/Multiplicative Decrease (AIMD) approach. This method involves gradually increasing the limit during normal process functioning but quickly reducing it when an issue (backoff event) occurs. There are various criteria for determining whether Gitaly is in trouble. In this proposal, we focus on two things:

- Lack of resources, particularly memory and CPU, which are essential for handling Git processes.
- Serious latency degradation.

The proposed solution is heavily inspired by many materials about this subject shared by folks from other companies in the industry, especially the following:

- TCP Congestion Control (RFC-2581, RFC-5681, RFC-9293, Computer Networks: A Systems Approach).
- Netflix adaptive concurrency limit (blog post and implementation)
- Envoy Adaptive Concurrency

(doc)

We cannot blindly apply a solution without careful consideration and expect it to function flawlessly. The suggested approach considers Gitaly's specific constraints and distinguishing features, including cgroup utilization and upload-pack RPC, among others.

The proposed solution does not aim to replace the existing limits in Gitaly for RPC concurrency
and pack object concurrency,
but automatically tweak the parameters. This means
that other aspects, such as queuing, in-queue timeout, queue length, partitioning, and scoping, will remain unchanged. The proposed solution only focuses on modifying the current value of the concurrency limit.

Design and implementation details

AIMD Algorithm

The Adaptive Concurrency Limit algorithm primarily uses the Additive Increase/Multiplicative Decrease (AIMD)
approach. This method involves gradually increasing the limit during normal process functioning but quickly reducing it when an issue occurs.

During initialization, we configure the following parameters:

- initialLimit: Concurrency limit to start with. This value is essentially equal to the current static concurrency limit.
- maxLimit: Maximum concurrency limit.
- minLimit: Minimum concurrency limit so that the process is considered as functioning. If it's equal to 0, it rejects all upcoming requests.
- backoffFactor: how fast the limit decreases when a backoff event occurs (0 < backoff < 1, default to 0.75)

When the Gitaly process starts, it sets limit = initialLimit, in which limit is the maximum in-flight requests allowed at a time.

Periodically, maybe once per 15 seconds, the value of the limit is re-calibrated:

- limit = limit + 1 if there is no backoff event since the last calibration. The new limit cannot exceed maxLimit.
- limit = limit backoffFactor otherwise. The new limit cannot be lower than minLimit.

When a process can no longer handle requests or will not be able to handle them soon, it is referred to as a back-off event. Ideally, we would love to see the efficient state as long as possible. It's the state where Gitaly is at its maximum capacity.

Ideally, min/max values are safeguards that aren't ever meant to be hit during operation, even overload. In fact, hitting either probably means that something is wrong and the dynamic algorithms aren't working well enough.

How requests are handled

The concurrency limit restricts the total number of in-flight requests (IFR) at a time.

- When IFR < limit, Gitaly handles new requests without waiting. After an increment, Gitaly immediately handles the subsequent request in the queue, if any.
- When IFR = limit, it means the limit is reached. Subsequent requests are queued, waiting for their turn. If the queue length reaches a configured limit, Gitaly rejects new requests immediately. When a request stays in the queue long enough, it is also automatically dropped by Gitaly.
- When IRF > limit, it's appropriately a consequence of backoff events. It means Gitaly handles more requests than the newly appointed limits. In addition to queueing upcoming requests similarly to the above case, Gitaly may start load-shedding in-flight requests if this situation is not resolved long enough.

At several points in time we have discussed whether we want to change queueing semantics. Right now we admit queued processes from the head of the queue (FIFO), whereas it was proposed several times that it might be preferable to admit processes from the back (LIFO).

Regardless of the rejection reason, the client received a ResourceExhausted response code as a signal that they would back off and retry later. Since most direct clients of Gitaly are internal, especially GitLab Shell and Workhorse, the actual users received some friendly messages. Gitaly can attach exponential pushback headers
to force internal clients to back off. However, that's a bit brutal and may lead to unexpected results. We can consider that later.

Backoff events

Each system has its own set of signals, and in the case of Gitaly, there are two aspects to consider:

- Lack of resources, particularly memory and CPU, which are essential for handling Git processes like git-pack-objects(1). When these resources are limited or depleted, it doesn't make sense for Gitaly to accept more requests. Doing so would worsen the saturation, and Gitaly addresses this issue by applying cgroups extensively. The following section outlines how accounting can be carried out using cgroup.
- Serious latency degradation. Gitaly offers various RPCs for different purposes besides serving Git data that is hard to reason about latencies. A significant overall latency decline is an indication that Gitaly should not accept more

requests. Another section below describes how to assert latency degradation reasonably.

Apart from the above signals, we can consider adding more signals in the future to make the system smarter. Some examples are Go garbage collector statistics, networking stats, file descriptors, etc. Some companies have clever tricks, such as using time drifting to estimate CPU saturation.

Backoff events of Upload Pack RPCs

Upload Pack RPCs and their siblings PackObjects RPC are unique to Gitaly. They are for the heaviest operations: transferring large volumes of Git data. Each operation may take minutes or even hours to finish. The time span of each operation depends on multiple factors, most notably the number of requested objects and the internet speed of clients.

Thus, latency is a poor signal for determining the backoff event. This type of RPC should only depend on resource accounting at this stage.

Backoff events of other RPCs

As stated above, Gitaly serves various RPCs for different purposes. They can also vary in terms of acceptable latency as well as when to recognize latency degradation. Fortunately, the current RPC concurrency limits implementation scopes the configuration by RPC and repository individually. The latency signal makes sense in this setting.

Apart from latency, resource usage also plays an important role. Hence, other RPCs should use both latency measurement and resource accounting signals.

Resource accounting with cgroup

The issue with saturation is typically not caused by Gitaly, itself but rather by the spawned Git processes that handle most of the work. These processes are contained within a cgroup,
and the algorithm for bucketing cgroup can be
found here.
Typically, Gitaly selects the appropriate cgroup for a request based on the target repository. There is also a parent cgroup to which all repository-level cgroups belong to.

Cgroup statistics are widely accessible. Gitaly can trivially fetch both resource capacity and current resource consumption via the following information in cgroup control file:

- memory.limitinbytes
- memory.usageinbytes
- cpu.cfsperiodus
- cpu.cfsquotaus
- cpuacct.usage

Fetching those statistics may imply some overheads. It's not necessary to keep them updated in real-time. Thus, they can be processed periodically in the limit adjustment cycle.

In the past, cgroup has been reliable in preventing spawned processes from exceeding their limits. It is generally safe to trust cgroup and allow processes to run without interference. However, when the limits set by cgroup are reached (at 100%), overloading can occur. This often leads to a range of issues such as an increase in page faults, slow system calls, memory allocation problems, and even out-of-memory kills. The consequences of such incidents are highlighted in
this example. Inflight requests are significantly impacted, resulting in unacceptable delays, timeouts, and even cancellations.

Besides, through various observations in the past, some Git processes such as git-pack-objects(1) build up memory over time. When a wave of git-pull(1) requests comes, the node can be easily filled up with various memory-hungry processes. It's much better to stop this accumulation in the first place.

As a result, to avoid overloading, Gitaly employs a set of soft limits, such as utilizing only 75% of memory capacity and 90% of CPU capacity instead of relying on hard limits. Once these soft limits are reached, the concurrency adjuster reduces the concurrency limit in a multiplicative manner. This strategy ensures that the node has enough headroom to handle potential overloading events.

In theory, the cgroup hierarchy allows us to determine the overloading status individually. Thus, Gitaly can adjust the concurrency limit for each repository separately. However, this approach would be unnecessarily complicated in practice. In contrast, it may lead to confusion for operators later.

As a good start, Gitaly recognizes an overloading event in either condition:

- Soft limits of the parent cgroup are reached.
- Soft limits of any of the repository cgroup are reached

It is logical for the second condition to be in place since a repository's capacity limit can be significant to the parent cgroup's capacity. This means that when the repository cgroup reaches its limit, fewer resources are available for other cgroups. As a result, reducing the concurrency limit delays the occurrence of overloading.

Latency measurement

When re-calibrate the concurrency limit, latency is taken into account for RPCs other than Upload Pack. Two things to consider when measuring latencies:

- How to record latencies
- How to recognize a latency degradation

It is clear that a powerful gRPC server such as Gitaly has the capability to

manage numerous requests per second per node. A production server can serve up to thousands of requests per second. Keeping track and storing response times in a precise manner is not practical.

The heuristic determining whether the process is facing latency degradation is interesting. The most naive solution is to pre-define a static latency threshold. Each RPC may have a different threshold. Unfortunately, similar to static concurrency limiting, it's challenging and tedious to pick a reasonable up-to-date value.

Fortunately, there are some famous algorithms for this line of problems, mainly applied in the world of TCP Congestion Control:

- Vegas Algorithm (CN: ASA - Chapter 6.4, Reference implementation)
- Gradient Algorithm (Paper, Reference implementation)

The two algorithms are capable of automatically determining the latency threshold without any pre-defined configuration. They are highly efficient and statistically reliable for real-world scenarios. In my opinion, both algorithms are equally suitable for our specific use case.

Load-shedding

Gitaly being stuck in the overloaded situation for too long can be denoted by two signs:

- A certain amount of consecutive backoff events
- More in-flight requests than concurrency limit for a certain amount of them

In such cases, a particular cgroup or the whole Gitaly node may become unavailable temporarily. In-flight requests are likely to either be canceled or timeout. On GitLab.com production, an incident is triggered and called for human intervention. We can improve this situation by load-shedding.

This mechanism deliberately starts to kill in-flight requests selectively. The main purpose is to prevent cascading failure of all inflight requests. Hopefully, after some of them are dropped, the cgroup/node can recover back to the normal situation fast without human intervention. As a result, it leads to net availability and resilience improvement.

Picking which request to kill is tricky. In many systems, request criticality is considered. A request from downstream is assigned with a criticality point. Requests with lower points are targeted first. Unfortunately, GitLab doesn't have a similar system. We have an
Urgency system,
but it is used for response time committing rather than criticality.

As a replacement, we can prioritize requests harming the system the most. Some criteria to consider:

- Requests consuming a significant percentage of memory
- Requests consuming a significant of CPU over time
- Slow clients
- Requests from IPs dominating the traffic recently
- In-queue requests/requests at an early stage. We don·t want to reject requests that are almost finished.

To get started, we can pick the first two criteria first. The list can be reinforced when learning from production later.

References

- Linkedin HODOR system
  - https://www.youtube.com/watch?v=-haM4ZpYNko
  - Hodor: Detecting and addressing overload in LinkedIn microservices
- https://www.linkedin.com/blog/engineering/infrastructure/hodor-overload-scenarios-and-the-evolution-of-their-detection-a
- Google SRE chapters about load balancing and overload:
  - https://sre.google/sre-book/load-balancing-frontend/
  - https://sre.google/sre-book/load-balancing-datacenter/
  - https://sre.google/sre-book/handling-overload/
  - https://sre.google/sre-book/addressing-cascading-failures/
  - https://sre.google/workbook/managing-load/
- Netflix Performance Under Load
- Netflix Adaptive Concurrency Limit
- Load Shedding with NGINX using adaptive concurrency control
- Overload Control for Scaling WeChat Microservices
- ReactiveConf 2019 - Jay Phelps: Backpressure: Resistance is NOT Futile
- AWS re:Invent 2021 - Keeping Netflix reliable using prioritized load shedding
- AWS Using load shedding to avoid overload
- "Stop Rate Limiting! Capacity Management Done Right" by Jon Moore
- Using load shedding to survive a success disaster·CRE life lessons
- Load Shedding in Web Services
- Load Shedding in Distributed Systems

SECTION: engineering/architecture/design-documents/gitaly_handle_upload_pack_in_http2_server/_index.md

{{< engineering/design-document-header >}}

Summary

All Git data transferring operations that use HTTP/SSH are handled by upload-pack RPCs in Gitaly.

These RPCs use a unique application-layer protocol called Sidechannel, which takes over the handshaking process during client dialing of the Gitaly gRPC server. This protocol allows for an out-of-band connection to transfer large data back to the client while

serving the gRPC connection as normal.

Although it provides a huge performance improvement over using pure gRPC streaming calls, this protocol is unconventional, confusing, sophisticated, and hard to extend or integrate into the next architecture of Gitaly. To address this, a new "boring" tech solution is proposed in this blueprint, which involves exposing a new HTTP/2 server to handle all upload-pack traffic.

Motivation

This section will explore how Git data transfers work, giving special attention to the role of Sidechannel optimization, and discussing both its advantages and disadvantages. Our main goal is to make Git data transfers simpler while maintaining the performance improvements that Sidechannel optimization has provided. We are looking for a solution that won't require us to completely rewrite the system and won't affect other parts of the systems and other RPCs.

How Git data transfer works

Skip this part if you are familiar with how Git data transfer architecture at GitLab.

Git data transfer is undeniably one of the crucial services that a Git server can offer. It is a fundamental feature of Git that was originally developed for Linux kernel development. As Git gained popularity, it continued to be recognized as a distributed system. However, the emergence of centralized Git services like GitHub or GitLab resulted in a shift in usage patterns. Consequently, handling Git data transfer in hosted Git servers has become challenging.

Git supports transferring data in packfiles over multiple protocols, notably HTTP and SSH. For more information, see pack-protocol and http-protocol.

In short, the general flow involves the following steps:

1. Reference Discovery: the server advertises its refs to the client.
1. Packfile negotiation: the client negotiates the packfile necessary for the transport with the server by sending the list of refs it "haves", "wants", etc.
1. Transfer Packfile data: the server composes the requested data and sends it back to the client using Pack protocol.

Further details and optimizations can underlie these steps. Git servers have never had issues with reference discovery and Packfile negotiation. The most demanding aspect of the process is the transfer of Packfile data, which is where the actual data exchange takes place.

For GitLab, Gitaly manages all Git operations. However, it is not accessible to external sources because Workhorse and GitLab Shell handle all external communications. Gitaly offers a gRPC server for them through specific RPCs such as SmartHTTP and SSH. In addition, it provides numerous other RPCs for GitLab Rails and other services. The following

diagram illustrates a clone using SSH:

mermaid
sequenceDiagram
  participant User as User
  participant UserGit as git fetch
  participant SSHClient as User's SSH Client
  participant SSHD as GitLab SSHD
  participant GitLabShell as gitlab-shell
  participant GitalyServer as Gitaly
  participant GitalyGit as git upload-pack

  User ->> UserGit: Runs git fetch
  UserGit ->> SSHClient: Spawns SSH client
  Note over User,SSHClient: On user's local machine

  SSHClient ->> SSHD: SSH session
  Note over SSHClient,SSHD: Session over Internet

  SSHD ->> GitLabShell: spawns gitlab-shell
  GitLabShell ->> GitalyServer: gRPC SSHUploadPack
  GitalyServer ->> GitalyGit: spawns git upload-pack

  Note over GitalyServer,GitalyGit: On Gitaly server
  Note over SSHD,GitalyGit: On GitLab server

Source: shameless copy from this doc

Git data transfer optimization with Sidechannel

In the past, we faced many performance problems when transferring huge amounts of data with gRPC. Epic 463 tracks
the work of this optimization. More context surrounds it but, in summary, there are two main problems:

- The way gRPC is designed is ideal for transmitting a large number of messages that are relatively small in size. However, when it comes to cloning a medium-sized
  repository, it can result in transferring gigabytes of data. Protobuf struggles when dealing with large data, as confirmed by various sources
  (examples: <https://protobuf.dev/programming-guides/techniques/large-data> and <https://github.com/protocolbuffers/protobuf/issues/7968>). It adds significant overhead
  and requires substantial CPU usage during encoding and decoding. Additionally, protobuf cannot read or write messages partially, which causes memory usage to skyrocket
  when the server receives multiple requests simultaneously.
- Secondly, grpc-go implementation also optimizes for a similar purpose. gRPC protocol is built on HTTP/2.
  When the gRPC server writes into the wire, it wraps the data inside an HTTP/2 data frame. grpc-go implementation maintains an asynchronous control buffer. It allocates
  new memory, copies the data over, and appends to the control buffer (client,

server). So, even if we can overpass the protobuf problem with a
custom codec, grpc-go is still an unsolved problem.
Upstream discussion (for read path only) about reusing memory is still pending.
An attempt to add pooled memory was reverted because it conflicts with typical
usage patterns.

We have developed a protocol called Sidechannel, which enables us to communicate raw binary data with clients through an out-of-band channel, bypassing the grpc-go implementation. For more information on Sidechannel, see this document. In summary, Sidechannel works
as follows:

- During the handshaking process of a gRPC server's acceptance of a client TCP connection, we use Yamux to multiplex the TCP
  connection at the application layer. This allows the gRPC server to operate on a virtual multiplexed connection called Yamux stream.
- When it becomes necessary for the server to transfer data, it establishes an additional Yamux stream. The data is transferred using that channel.

mermaid
sequenceDiagram
    participant Workhorse
    participant Client handshaker
    participant Server handshaker
    participant Gitaly
    Note over Workhorse,Client handshaker: Workhorse process
    Note over Gitaly,Server handshaker: Gitaly process
    Workhorse ->> Client handshaker: -
    Client handshaker ->> Server handshaker: 1. Dial
    Note over Server handshaker: Create Yamux session
    Server handshaker ->> Client handshaker: -
    Client handshaker ->> Workhorse: Yamux stream 1
    Note over Workhorse: Persist Yamus session

    par Yamux stream 1
        Workhorse ->>+ Gitaly: 2. PostUploadWithSidechannel
        Note over Gitaly: Validate request
        Gitaly ->> Workhorse: 3. Open Sidechannel
    end

    Note over Workhorse: Create Yamux stream 2

    par Yamux stream 2
        Workhorse ->> Gitaly: 4. Packfile Negotiation and stuff
        Workhorse ->> Gitaly: 5. Half-closed signal
        Gitaly ->> Workhorse: 6. Packfile data
        Note over Workhorse: Close Yamux stream 2
    end

    par Continue Yamux stream 1

```
    Gitaly ->>- Workhorse: 7. PostUploadPackWithSidechannelResponse
  end
```

Sidechannel solved the original problem for us so far. We observed a huge improvement in Git transfer utilization and a huge reduction in CPU and memory usage. Of course, it comes with some tradeoffs:

- It's a clever trick, maybe too clever. grpc-go provides a handshaking hook solely for authentication purposes. It's not supposed to be used for connection modification.
- Because Sidechannel works on the connection level, all RPCs using that connection must establish the multiplexing, even if they are not targeted upload-pack ones.
- Yamux is an application-level multiplexer. It's heavily inspired by HTTP/2, especially the binary framing, flow control, etc. We can call it a slimmed-down version of HTTP/2.
  We are stacking two framing protocols when running gRPC on top of Yamux.
- The detailed implementation of Sidechannel is sophisticated. Apart from the aforementioned handshaking, when the server dials back, the client must start an invisible
  gRPC server for handshaking before handing it back to the handler. It also implements an asymmetric framing protocol inspired by pktline. This protocol is tailored
  for upload-pack RPC and overcomes the lack of the "half-closed" ability of Yamux.

All of those complexity adds up to the point that it becomes a burden for the maintenance and future evolution of Gitaly. When looking forward to the future Gitaly
Raft-based architecture, Sidechannel is usually a puzzle. Reasoning about the routing strategies and implementations must consider the compatibility with Sidechannel.
Sidechannel is an application-layer protocol so most client-side routing libraries don't work well with it. Besides, we must ensure the performance
gained by Sidechannel is reserved. Eventually, we can still find a way to fit Sidechannel in, but the choices are pretty limited, likely leading to another
clever hack.

Replacing Sidechannel with a simpler and widely adopted technology that maintains the same performance characteristics would be beneficial. One potential solution to address
all the issues would be to transfer all upload-pack RPCs to a pure HTTP/2 server.

Goals

- Find an alternative to Sidechannel for upload-pack RPCs that is easier to use, uncomplicated, and widely adopted.
- The implementation must use supported API and use cases of popular libraries, frameworks, or Go's standard lib. No more hacking in the transportation layer.
- The new solution must work well with Praefect and be friendly to future routing mechanisms, load-balancers, and proxies.
- Have the same performance and low resource utilization as Sidechannel.
- Allow gradual rollout and full backwards compatibility with clients.

Non-Goals

- Re-implement everything we invested into gRPC, such as authentication, observability, concurrency limiting, metadata propagation, etc. We don't want to maintain or

replicate features between two systems simultaneously.
- Modify other RPCs or change how they are used in clients.
- Migrate all RPCs to a new HTTP/2 server.

Proposal

A huge portion of this blueprint describes the historical contexts and why we should move on. The proposed solution is straightforward: Gitaly exposes a pure HTTP2 server to handle all upload-pack RPCs. Other RPCs stay intact and are handled by the existing gRPC server.

```
mermaid
flowchart LR
   Workhorse-- Other RPCs --> Port3000{{":3000"}}
   Port3000 --o gRPCServer("gRPC Server")
   Workhorse-- PostUploadPack --> Port3001{{":3001"}}
   Port3001 --o HTTP2Server("HTTP/2 Server")
   GitLabShell("GitLab Shell")-- SSHUploadPack --> Port3001
   GitLabRails("GitLab Rails") -- Other RPCs --> Port3000
   subgraph Gitaly
     Port3000
     gRPCServer
     Port3001
     HTTP2Server
   end
```

As mentioned earlier, Sidechannel utilizes the Yamux multiplexing protocol, which can be seen as a streamlined version of HTTP/2. When HTTP/2 is used instead, the core functionality remains unchanged, namely multiplexing, binary framing protocol, and flow control. This means that clients like Workhorse can efficiently exchange large amounts of binary data over the same TCP connection without requiring a custom encoding and decoding layer like Protobuf. This was the intended use case for HTTP/2 from the start and, in theory, it can deliver the same level of performance as Sidechannel. Moreover, this repla